

ASSIGNMENT REPORT

Digital Design

Assignment 5.1 - Automatic Garage Door Controller (Moore FSM)

Course	Digital Design
Assignment	Assignment 5.1 - Automatic Garage Door Controller
Student Name	Mohamed Hossam El-Sawy
Instructor / TA	Eng. Ali Temsah
Submission Date	July 26, 2026

01 OBJECTIVE

This assignment designs, implements and verifies an Automatic Garage Door Controller as a Moore finite state machine (FSM). The controller drives an Up motor and a Down motor from two limit-sensor inputs and a single user push-button, and is initialised to a safe IDLE state through an asynchronous reset.

02 PROBLEM STATEMENT / REQUIREMENTS

A Moore FSM must be designed and verified with the following behaviour:

- Three inputs: Activate (user push-button), UP_Max (sensor, high when the door is fully open), DN_Max (sensor, high when the door is fully closed).
- Two outputs: UP_M (drives the Up motor) and DN_M (drives the Down motor).
- The door is always either fully open or fully closed - Activate triggers the Up motor when the door is down, and the Down motor when the door is up.
- The FSM is initialised to the IDLE state using an asynchronous reset (RST).
- Three states: IDLE (default/reset state, both motors off), Mv_Up (Up motor enabled to open the door), Mv_Dn (Down motor enabled to close the door).
- Deliverables: the Verilog implementation of the Moore FSM, and a self-checking testbench that validates the design at a 50 MHz clock frequency.

03 DESIGN & ANALYSIS

3.1 Design Approach

The controller is coded as a classic two-always-block Moore FSM. `current_state` is a 2-bit register updated on posedge `CLK`, with an asynchronous active-low reset (negedge `RST` in the sensitivity list) that forces `current_state` back to `IDLE_state` regardless of the clock. A second, purely combinational always block computes `next_state` from `current_state` and the inputs using a case statement. Because this is a Moore machine, the outputs `UP_M` and `DN_M` are continuous assign statements that depend only on `current_state` (`UP_M = current_state == Mv_Up_state`, `DN_M = current_state == Mv_Dn_state`) - they never depend directly on the inputs, so they only change on a clock edge together with the state.

From `IDLE`, the design interprets which sensor is already asserted to decide direction: if the door is closed (`DN_Max=1`, `UP_Max=0`) and `Activate` is pressed, the FSM opens the door via `Mv_Dn/Mv_Up` encoding matched to the state diagram; if the door is open (`UP_Max=1`, `DN_Max=0`) and `Activate` is pressed, it moves the other way. If both sensors read the same value (a broken/ambiguous sensor condition) the FSM stays safely in `IDLE`. Once moving, `Mv_Up` and `Mv_Dn` ignore `Activate` and only return to `IDLE` when their respective limit sensor (`UP_Max` or `DN_Max`) is reached.

3.2 State Tables

Moore output table - `UP_M/DN_M` depend only on the present state:

State	Encoding	UP_M	DN_M
IDLE	00	0	0
Mv_Up	01	1	0
Mv_Dn	10	0	1

Table 1: Moore output table for Automatic_Garage_Door_Controller

Next-state table:

Current State	Condition	Next State
IDLE	Activate = 0	IDLE
IDLE	Activate = 1, UP_Max = 1, DN_Max = 0	Mv_Dn
IDLE	Activate = 1, UP_Max = 0, DN_Max = 1	Mv_Up
IDLE	Activate = 1, UP_Max = DN_Max (both sensors agree/broken)	IDLE
Mv_Up	UP_Max = 0	Mv_Up (hold)
Mv_Up	UP_Max = 1	IDLE
Mv_Dn	DN_Max = 0	Mv_Dn (hold)
Mv_Dn	DN_Max = 1	IDLE

Table 2: Next-state transition table

Top-level port list:

Signal Name	Direction	Width (bits)	Description
UP_Max	input	1	Sensor - high when the door is completely open
DN_Max	input	1	Sensor - high when the door is completely closed
Activate	input	1	User push-button
CLK	input	1	System clock
RST	input	1	Asynchronous active-low reset
UP_M	output	1	Drives the Up motor
DN_M	output	1	Drives the Down motor

Table 3: Automatic_Garage_Door_Controller port description

3.3 Block Diagram and State Diagram

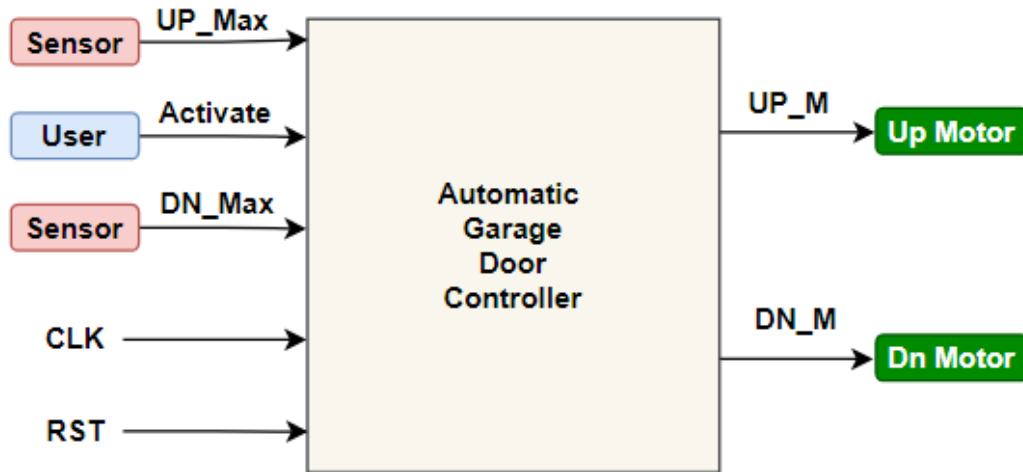


Figure 1: Automatic Garage Door Controller block interface

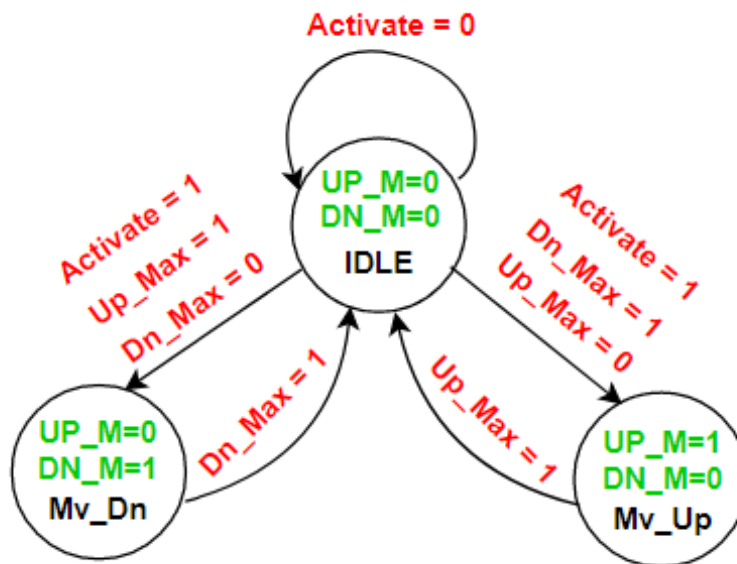


Figure 2: Moore state diagram - IDLE, Mv_Up and Mv_Dn states

3.4 HDL Implementation

Automatic_Garage_Door_Controller.v - full source (state register, asynchronous reset, Moore outputs and next-state logic):

```
module Automatic_Garage_Door_Controller (
    input UP_Max,DN_Max,
    input Activate,
    input CLK,RST,
    output UP_M,DN_M
);
    localparam [1:0] IDLE_state = 0,
                    Mv_Up_state = 1,
                    Mv_Dn_state = 2;
    reg [1:0] next_state,current_state;
    always @(posedge CLK or negedge RST) begin
        if (!RST) begin
            current_state <= IDLE_state;
        end
    end
endmodule
```

```

        end
        else
            current_state <= next_state;
        end
    assign UP_M = current_state == Mv_Up_state;
    assign DN_M = current_state == Mv_Dn_state;
    always @(*) begin
        next_state = IDLE_state;
        case (current_state)
            IDLE_state:
                if (!Activate) begin
                    next_state = IDLE_state;
                end
                else if (UP_Max && !DN_Max) begin
                    next_state = Mv_Dn_state;
                end
                else if (!UP_Max && DN_Max) begin
                    next_state = Mv_Up_state;
                end
                else
                    next_state = IDLE_state;
            Mv_Up_state:
                if (UP_Max) begin
                    next_state = IDLE_state;
                end
                else
                    next_state = Mv_Up_state;
            Mv_Dn_state:
                if (DN_Max) begin
                    next_state = IDLE_state;
                end
                else
                    next_state = Mv_Dn_state;
        endcase
    end
endmodule //Automatic_Garage_Door_Controller

```

04 SIMULATION & RESULTS

Verification approach: Automatic_Garage_Door_Controller_TB.v is a self-checking testbench that drives UP_Max, DN_Max and Activate on the negedge of a 50 MHz clock (CLK_PERIOD = 20 ns) and checks DUT.current_state, UP_M and DN_M on the following negedge against expected values, reporting PASS/FAIL through reusable apply_reset / drive_inputs / check_state_outputs tasks. The suite covers 19 test cases: asynchronous reset, IDLE hold, both direction transitions (open/close), holding state while a sensor is still low, returning to IDLE once a limit sensor is reached, IDLE holding when both sensors read the same (ambiguous) value, Mv_Up/Mv_Dn correctly ignoring Activate while moving, and asynchronous reset asserted mid-cycle from both Mv_Up and Mv_Dn.

4.1 Test Case Summary

#	Test Case	Time (ns)	Result
1	Async Reset -> IDLE	60000	PASSED
2	IDLE hold (Activate=0)	100000	PASSED
3	IDLE -> Mv Up (open door)	140000	PASSED
4	Mv_Up hold (UP_Max=0)	180000	PASSED
5	Mv_Up -> IDLE (UP_Max=1)	220000	PASSED
6	IDLE -> Mv Dn (close door)	260000	PASSED
7	Mv_Dn hold (DN_Max=0)	300000	PASSED
8	Mv_Dn -> IDLE (DN_Max=1)	340000	PASSED
9	IDLE hold (both sensors low)	380000	PASSED

#	Test Case	Time (ns)	Result
10	IDLE hold (both sensors high)	420000	PASSED
11	Enter Mv Up for ignore test	460000	PASSED
12	Mv Up ignores Activate	500000	PASSED
13	Return to IDLE	540000	PASSED
14	Enter Mv Dn for ignore test	580000	PASSED
15	Mv Dn ignores Activate	620000	PASSED
16	Async Reset from Mv Dn	666000	PASSED
17	IDLE cleanup after async reset	700000	PASSED
18	Enter Mv Up for reset test	740000	PASSED
19	Async Reset from Mv Up	766000	PASSED

Table 4: Summary of all 19 directed test cases (extracted from the simulation log) - 19/19 PASSED

4.2 Simulation Waveform

ModelSim waveform showing current_state/next_state, UP_Max, DN_Max, Activate, CLK, RST and the UP_M/DN_M outputs cycling through IDLE -> Mv_Up -> IDLE -> Mv_Dn -> IDLE as Activate and the limit sensors are exercised:

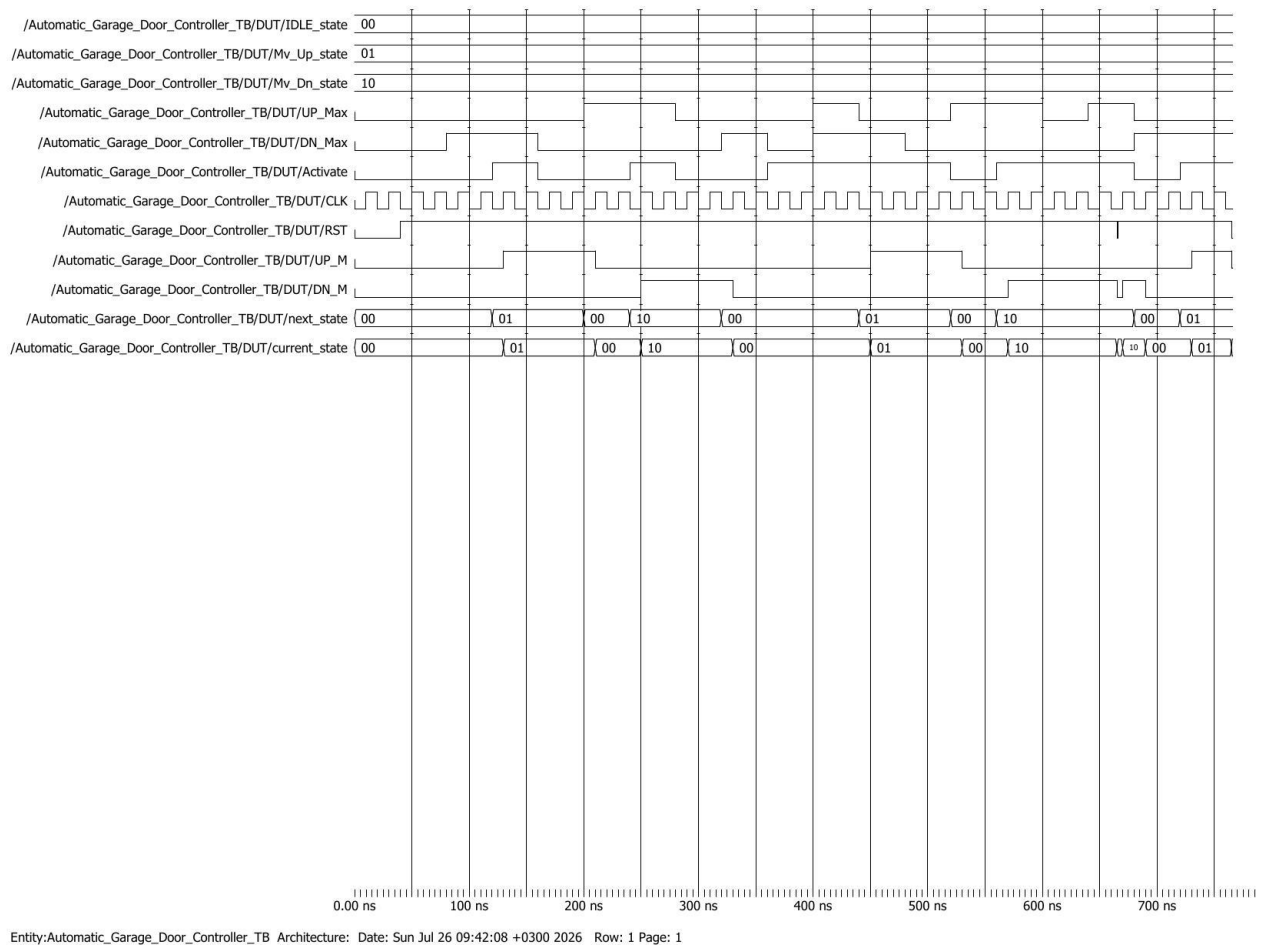


Figure 3: ModelSim simulation waveform for Automatic_Garage_Door_Controller_TB

4.3 Full Simulation Log

Console transcript produced by Automatic_Garage_Door_Controller_TB.v, showing every PASS result and the final tally (19/19 passed, 0 failed):

```
=====
Automatic Garage Door Controller - Self-Checking Testbench
=====
[INFO] Clock Period = 20 ns (50 MHz)

[INFO] Applying asynchronous reset...
TEST 1 PASSED: Async Reset -> IDLE @ 60000
TEST 2 PASSED: IDLE hold (Activate=0) @ 100000
TEST 3 PASSED: IDLE -> Mv_Up (open door) @ 140000
TEST 4 PASSED: Mv_Up hold (UP_Max=0) @ 180000
TEST 5 PASSED: Mv_Up -> IDLE (UP_Max=1) @ 220000
TEST 6 PASSED: IDLE -> Mv_Dn (close door) @ 260000
TEST 7 PASSED: Mv_Dn hold (DN_Max=0) @ 300000
TEST 8 PASSED: Mv_Dn -> IDLE (DN_Max=1) @ 340000
TEST 9 PASSED: IDLE hold (both sensors low) @ 380000
TEST 10 PASSED: IDLE hold (both sensors high) @ 420000
TEST 11 PASSED: Enter Mv_Up for ignore test @ 460000
TEST 12 PASSED: Mv_Up ignores Activate @ 500000
TEST 13 PASSED: Return to IDLE @ 540000
TEST 14 PASSED: Enter Mv_Dn for ignore test @ 580000
TEST 15 PASSED: Mv_Dn ignores Activate @ 620000
TEST 16 PASSED: Async Reset from Mv_Dn @ 660000
TEST 17 PASSED: IDLE cleanup after async reset @ 700000
TEST 18 PASSED: Enter Mv_Up for reset test @ 740000
TEST 19 PASSED: Async Reset from Mv_Up @ 766000

=====
TEST SUMMARY
=====
Total Tests Run : 19
Passed : 19
Failed : 0
-----
RESULT : ALL TESTS PASSED
=====
** Note: $finish : Automatic_Garage_Door_Controller_TB.v(287)
Time: 766 ns Iteration: 0 Instance: /Automatic_Garage_Door_Controller_TB
```

Log 1: Full ModelSim transcript for Automatic_Garage_Door_Controller_TB

05 DISCUSSION

All 19 directed test cases passed. The asynchronous reset behaves correctly both at power-on and when asserted mid-cycle from Mv_Up and Mv_Dn (Tests 16 and 19), confirming current_state returns to IDLE_state independently of CLK as required. The Moore-output property was specifically exercised by holding Mv_Up/Mv_Dn with Activate still asserted (Tests 12 and 15): UP_M and DN_M remained correct and unaffected by the stale Activate input, since they are combinationaly tied only to current_state. The ambiguous-sensor cases (Tests 9 and 10, both sensors low or both high) confirm the FSM fails safe by remaining in IDLE rather than committing to a direction it cannot verify. No mismatches or timing violations were observed over the 766 ns simulation.

06 CONCLUSION

A Moore FSM was designed and verified for the Automatic Garage Door Controller, with three states (IDLE, Mv_Up, Mv_Dn), asynchronous active-low reset, and outputs registered purely as a function of state. A 19-case self-checking testbench running at 50 MHz confirmed correct state transitions in both directions, correct holding behaviour while a door is mid-travel, correct handling of ambiguous sensor readings, correct ignoring of the Activate input while moving, and correct asynchronous reset recovery from any state.